Part II: Examples

1.1 Basic Elements of a Class Declaration
1.2 Static Members in Class Declaration
1.3 Defining a Subclass
1.4 Using Aggregation
1.5 A Sample Program
1.6 A Single-File Source-Code Program
2.1 Evaluation Order of Operands and Arguments
2.2 Numeric Promotion in Arithmetic Expressions
2.3 Short-Circuit Evaluation Involving Conditional Operators
2.4 Bitwise Operations
3.1 Default Values for Fields
3.2 Flagging Uninitialized Local Variables of Primitive Data Types
3.3 Flagging Uninitialized Local Reference Variables
3.4 Using the `this` Reference
3.5 Namespaces
3.6 Accessing Static Members in a Class
3.7 Using Arrays
3.8 Using Anonymous Arrays
3.9 Using Multidimensional Arrays
3.10 Passing Primitive Values
3.11 Passing Reference Values
3.12 Passing Arrays
3.13 Array Elements as Primitive Data Values
3.14 Array Elements as Reference Values
3.15 Calling a Variable Arity Method
3.16 Passing Program Arguments
3.17 Illustrating Local Variable Type Reference
4.1 Fall-Through in a `switch` Statement with the Colon Notation
4.2 Nested `switch` Statements with the Colon Notation
4.3 Nested `switch` Statements with the Arrow Notation
4.4 Strings in a `switch` Statement
4.5 Enums in a `switch` Statement
4.6 A `yield` Statement in a `switch` Expression with the Colon Notation

4.7 Statement Blocks in a switch Expression with the Arrow Notation
4.8 Expression Actions in a switch Expression with the Arrow Notation
4.9 Returning Multiple Values as a Record from a switch Expression
4.10 The break Statement
4.11 Labeled break Statement
4.12 The continue Statement
4.13 Labeled continue Statement
4.14 The return Statement
5.1 Extending Classes: Inheritance and Accessibility
5.2 Overriding, Overloading, and Hiding
5.3 Ambiguous Call to Overloaded Methods
5.4 Using the super Keyword
5.5 Constructor Overloading
5.6 The this() Constructor Call
5.7 The super() Constructor Call
5.8 Using Abstract Classes
5.9 Using the final Modifier
5.10 Implementing Interfaces
5.11 Default Methods in Interfaces
5.12 Inheriting Default Method Implementations from Superinterfaces
5.13 Inheriting Method Implementations from Supertypes
5.14 Static Methods in Interfaces
5.15 Private Methods in Interfaces
5.16 Constants in Interfaces
5.17 Inheriting Constants in Interfaces
5.18 Assigning and Passing Reference Values
5.19 Choosing the Most Specific Method (Simple Case)
5.20 Overloaded Method Resolution
5.21 The instanceof Type Comparison and Cast Operators
5.22 Using the instanceof Type Comparison Operator
5.23 Using Polymorphism
5.24 Using Enums
5.25 Declaring Enum Constructors and Members
5.26 Declaring Constant-Specific Class Bodies
5.27 The CD Class
5.28 The CD Record Class
5.29 Normal Canonical Record Constructor
5.30 Compact Record Constructor and Non-Canonical Constructor
5.31 Other Member Declarations in a Record Class
5.32 Declaring sealed , non-sealed , and final Classes

5.33 Using Sealed Classes
6.1 Defining Packages and Using Type Import
6.2 Single Static Import
6.3 Avoiding the Interface Constant Antipattern
6.4 Importing Enum Constants
6.5 Shadowing Static Import
6.6 Conflict in Importing a Static Method with the Same Signature
6.7 Access Modifiers for Top-Level Reference Types
6.8 Accessibility of Class Members
6.9 Class Scope
6.10 Implementing an Immutable Class
7.1 Method Execution
7.2 The `try-catch` Construct
7.3 Exception Propagation
7.4 The `try-catch-finally` Construct
7.5 The `try-finally` Construct
7.6 The `finally` Clause and the `return` Statement
7.7 Throwing Exceptions Programmatically
7.8 The `throws` Clause
7.9 Using Multiple `catch` Clauses
7.10 Using the Multi-`catch` Clause
7.11 Precise Rethrowing of Exceptions
7.12 Precise `throws` Clause
7.13 Conditions for Rethrowing Final Exceptions
7.14 Chaining Exceptions
7.15 Naive Resource Management
7.16 Explicit Resource Management
7.17 Using `try` with Automatic Resource Management (ARM)
7.18 Implementing the `AutoCloseable` Interface
8.1 Methods in the `Object` Class
8.2 Text Representation of Integers
8.3 String Construction and Internment
8.4 Reading Characters from a `String`
8.5 Controlling String Builder Capacity
9.1 Overview of Type Declarations
9.2 Static Member Types
9.3 Importing Static Member Types
9.4 Accessing Members in Enclosing Context (Static Member Classes)
9.5 Defining and Instantiating Non-Static Member Classes
9.6 Qualified `this` and Qualified Class Instance Creation Expression

9.7 Inheritance Hierarchy and Enclosing Context
9.8 Extending Inner Classes
9.9 Declaring Local Classes
9.10 Accessing Local Declarations in the Enclosing Block (Local Classes)
9.11 Accessing Members in the Enclosing Class (Local Classes)
9.12 Instantiating Local Classes
9.13 Defining Static Local Record Classes
9.14 Defining Anonymous Classes
9.15 Accessing Declarations in Enclosing Context (Anonymous Classes)
10.1 Initializer Expression Order and Method Calls
10.2 Exceptions in Initializer Expressions
10.3 Static Initializers and Forward References
10.4 Static Initializers and Order of Execution
10.5 Static Initializer Blocks and Exceptions
10.6 Instance Initializers and Forward References
10.7 Instance Initializer Block in Anonymous Class
10.8 Exception Handling in Instance Initializer Blocks
10.9 Object State Construction
10.10 Initialization Anomaly under Object State Construction
11.1 A Legacy Class
11.2 A Generic Class for Nodes
11.3 A Generic Interface and Its Implementation
11.4 A Non-Generic Class Implementing a Generic Interface
11.5 Extending Generic Types
11.6 Unchecked Warnings
11.7 Illustrating Get and Set Operations Using Parameterized References
11.8 Implementing the `Comparable<E>` Interface
11.9 Declaring Generic Methods
11.10 Implementing a Simplified Generic Stack
11.11 Using Recursive Bounds
11.12 Using the `@Override` Annotation
11.13 Subsignatures
11.14 Overriding from Generic Supertype
11.15 Missing Supertype Parameterization
11.16 Genericity Cannot be Added to Inherited Methods
11.17 Restrictions on Exception Handling
11.18 Type Parameter in `throws` Clause
11.19 Generic Nested Types
11.20 Instantiating Generic Nested Classes
11.21 Enum Implements Parameterized Generic Interface

12.1 Using an ArrayList
13.1 Accessing Members in the Enclosing Object
13.2 Accessing Local Variables in the Enclosing Method
13.3 Filtering an ArrayList<E>
13.4 Implementing Suppliers
13.5 Implementing the BiPredicate<T, U> Functional Interface
13.6 Implementing Functions
14.1 Not Overriding the Object.equals() Method
14.2 Implications of Not Overriding the Object.equals() Method
14.3 Implementing the equals() Method
14.4 Implications of Implementing the equals() Method
14.5 Implications of Not Overriding the Object.hashCode() Method
14.6 Implementing the hashCode() Method
14.7 Implications of Implementing the hashCode() Method
14.8 Implementing the compareTo() Method of the Comparable<E> Interface
14.9 Implications of Implementing the compareTo() Method
14.10 Natural Ordering and Total Orderings
14.11 Using a Comparator for Version Numbers
15.1 Iterating Over a Collection
15.2 Converting Collections to Arrays
15.3 Using Lists
15.4 Iterating Over Sets
15.5 Using Sets
15.6 Using Navigable Sets
15.7 Using Priority Queues
15.8 Demonstrating Deque Operations
15.9 Using Deques as a LIFO Stack and as a FIFO Queue
15.10 Using Maps
15.11 Using Navigable Maps
16.1 The CD Example Classes
16.2 Data Processing Using Streams
16.3 Stream Mapping and Loop Fusion
16.4 Order of Intermediate Operations
16.5 Filtering
16.6 Examining Stream Elements
16.7 Sorting Streams
16.8 Using Optionals
16.9 Using Numerical Optionals
16.10 Identity Matrix Test
16.11 Implementing Functional Reductions

16.12 Implementing Mutable Reductions
16.13 Flat mapping
16.14 Benchmarking
17.1 Creating Local Dates and Local Times
17.2 Using Local Dates and Local Times
17.3 Temporal Arithmetic
17.4 Valid Temporal Units and Temporal Fields
17.5 Period-Based Loop
17.6 More Temporal Arithmetic
17.7 Duration-Based Loop
17.8 Flight Time Information
17.9 Adjusting for DST Crossovers
17.10 Converting to Legacy Date
18.1 Understanding Locales
18.2 The `BasicResources` Bundle Family (See Also Example 18.3)
18.3 Using Resource Bundles (See Also Example 18.2)
18.4 Locating Resource Bundles
18.5 Rounding Modes
18.6 Using the `DecimalFormat` class
18.7 Formatting and Parsing with Letter Patterns
18.8 Formatting Compound Messages
18.9 Using Choice Pattern
18.10 Using the `ChoiceFormat` Class
19.1 Source Code Files for the `adviceApp` Application
19.2 Selected Source Code Files in the `adviceOpen` Application
19.3 Specifying the `Advice` Service Interface
19.4 Implementing the Advice Service Providers
19.5 Implementing the Advice Service Locator
19.6 Implementing of the Advice Service Consumer
20.1 Copying a File Using a Byte Buffer
20.2 Reading and Writing Binary Values
20.3 Copying a File Using a Character Buffer
20.4 Demonstrating Readers and Writers, and Character Encoding
20.5 Changing Passwords
20.6 Object Serialization
20.7 Non-Serializable Objects
20.8 Customized Serialization
20.9 Serialization and Inheritance
20.10 Class Versioning
21.1 Querying `Path` Objects

21.2 Reading and Writing Text Files
21.3 Reading and Writing Bytes
21.4 Accessing Individual Attributes
21.5 Traversing the Directory Hierarchy
22.1 Implementing the `Runnable` Interface
22.2 Extending the `Thread` Class
22.3 Thread States
22.4 Mutual Exclusion
22.5 Interrupt Handling
22.6 Waiting and Notifying
22.7 Joining of Threads
22.8 Thread Termination
22.9 Deadlock
22.10 Livelock
22.11 Starvation
23.1 Executor Lifecycle
23.2 Submitting Tasks and Handling Task Results
23.3 Executing Scheduled One-Shot Tasks
23.4 Executing Scheduled Periodic Tasks
23.5 Task Cancellation
23.6 Filtering Values Using Fork/Join Framework
23.7 Testing Counter Implementations
23.8 Visibility of Shared Data
23.9 Volatile Counter
23.10 Atomic Counter
23.11 Synchronized Counter
23.12 Reentrant Lock Counter
23.13 Reentrant Read-Write Lock Counter
23.14 Cyclic Barrier
23.15 Count-Down Latch
23.16 Serial Access in Synchronized Views of Collections
23.17 Compound Mutually Exclusive Operation
23.18 Concurrent Collections
23.19 Using a Concurrent Map
23.20 Linked Blocking Queue
23.21 Copy-on-Write Array List
24.1 Connecting to the `musicDB` Database
24.2 Executing a Prepared Statement
24.3 Prepared Statement to Execute `INSERT`, `UPDATE`, and `DELETE`
24.4 Stored Procedures and Functions

24.5 Processing a ResultSet
24.6 Using ResultSet Options
24.7 Discovering Metadata for the Database and ResultSet
24.8 Controlling Transactions
25.1 Using the @Deprecated Annotation
25.2 Using the @SuppressWarnings Annotation
25.3 Using the @SafeVarargs Annotation
25.4 Processing Annotations
25.5 Annotation Processor

Previous chapter
< Part II: Tables

Next chapter
Foreword >